

2 Computer Architecture

Hui Xu, xuh@fudan.edu.cn

Learning Objective:

- Understand the main components of a computer.
- Be able to read simple assembly instructions and understand how a computer works.

2.1 Overview

The previous chapter discussed how arithmetic circuits work. They are core components of CPUs. This chapter further explores how CPUs work. Before that, we will first provide an overview of computer architecture.

A computer is generally composed of a CPU, memory, input and output devices, storage, and network devices (Figure 2.1).

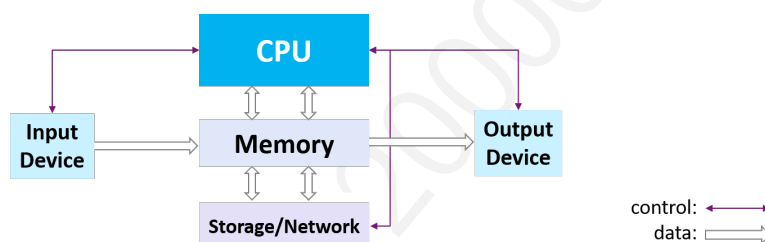


Figure 2.1: Main components of a computer.

- *CPU (Central Processing Unit)*: Acts as the brain of the computer by fetching instructions from memory, decoding and executing them, performing computations in the Arithmetic Logic Unit (ALU), and coordinating data transfers between the CPU, memory, storage, and input/output devices.
- *Memory*: Temporarily holds data and programs for fast access by the CPU.
- *Input devices*: Bring data and instructions into the system from the external world, such as keyboards, touchscreens, microphones, and cameras.
- *Output devices*: Present results and processed information to the user, such as monitors, speakers, and printers.
- *Storage*: Provides persistent storage for data and programs, even when the computer is turned off. It serves as both an input and an output device.
- *Network*: Enables communication and data exchange between computers and other devices over local or wide-area networks.

These components communicate through buses, such as the control bus, which coordinates signals, and the data bus, which transfers data. Next, we will discuss how a CPU works and interacts with other components.

2.2 Central Processing Unit (CPU)

The CPU executes instructions from programs stored in memory, performing arithmetic, logical, control, and memory access operations. It consists of several key components:

- *Arithmetic Logic Unit (ALU)*: Performs arithmetic and logical operations on data.
- *Control Unit (CU)*: Controls the execution of instructions by fetching and decoding instructions and coordinating their execution.
- *Load/Store Unit (LSU)*: Handles memory access operations by loading data from memory into registers and storing data from registers back into memory.
- *Registers*: Small, high-speed storage locations inside the CPU for temporarily holding operands, results, and addresses.

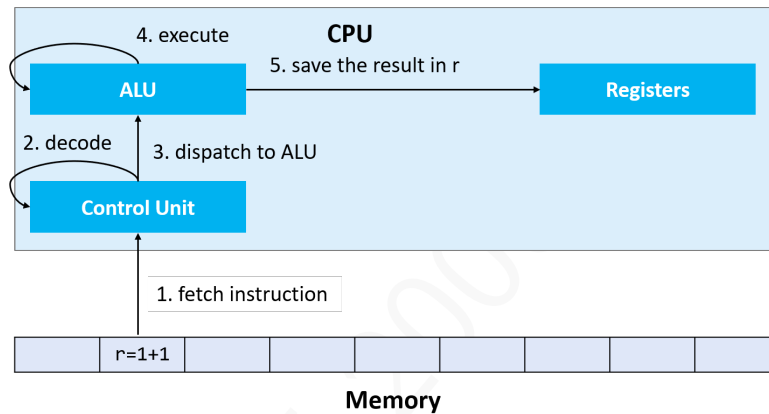


Figure 2.2: Executing the instruction $r=1+1$.

Figure 2.2 illustrates how these components work together to execute an instruction. The execution of instructions follows a sequence known as the fetch–decode–execute cycle:

- *Fetch*: The control unit retrieves the binary instruction bits from memory.
- *Decode*: The control unit interprets the instruction to determine what operation should be performed and which operands and registers are involved.
- *Execute*: The instruction is carried out. Arithmetic and logical operations are performed by the ALU, while memory access operations are handled by the LSU. The results or other effects of the instruction are then produced as required.

Since this instruction uses immediate values as operands and does not involve memory access, the LSU is not used. Next, we will discuss more cases involving memory access.

2.3 Memory

Memory in a computer generally refers to RAM (Random-Access Memory), a type of volatile memory that allows data to be read and written using memory addresses. Each memory address typically identifies one byte (8 bits), which is the smallest addressable unit of memory in most systems. The term “random access” means that any memory location can be accessed directly using its address, rather than having to access memory locations sequentially.

Now we extend the integer addition instruction in Figure 2.2 to the addition of two variables (*i.e.*, $z = x + y$) whose values are stored in memory. This operation can be compiled into four instructions in our abstract instruction set, as shown in Code 2.1.

```

r1 = load x      ; load the value of x from memory into register r1.
r2 = load y      ; load the value of y from memory into register r2.
r2 = add r1, r2  ; add the values in r1 and r2 and save the result in r2.
store r2, z      ; store the value of r2 into memory at the address of z.

```

Code 2.1: Instructions for $z=x+y$.

Note that the identifiers x , y , and z do not exist in the actual machine instructions. Instead, they are replaced with the corresponding memory addresses. Figure 2.3 illustrates a sample layout of these instructions and variables, as well as the execution steps. In this example, the three variables are stored at memory addresses $0x18$, $0x1B$, and $0x20$, respectively. The first instruction, $r1 = \text{load } x$, is located at memory address $0x04$. After executing this instruction, the program counter (PC) in the CPU points to the next instruction at address $0x08$. The CPU then uses the PC to fetch and execute the next instruction.

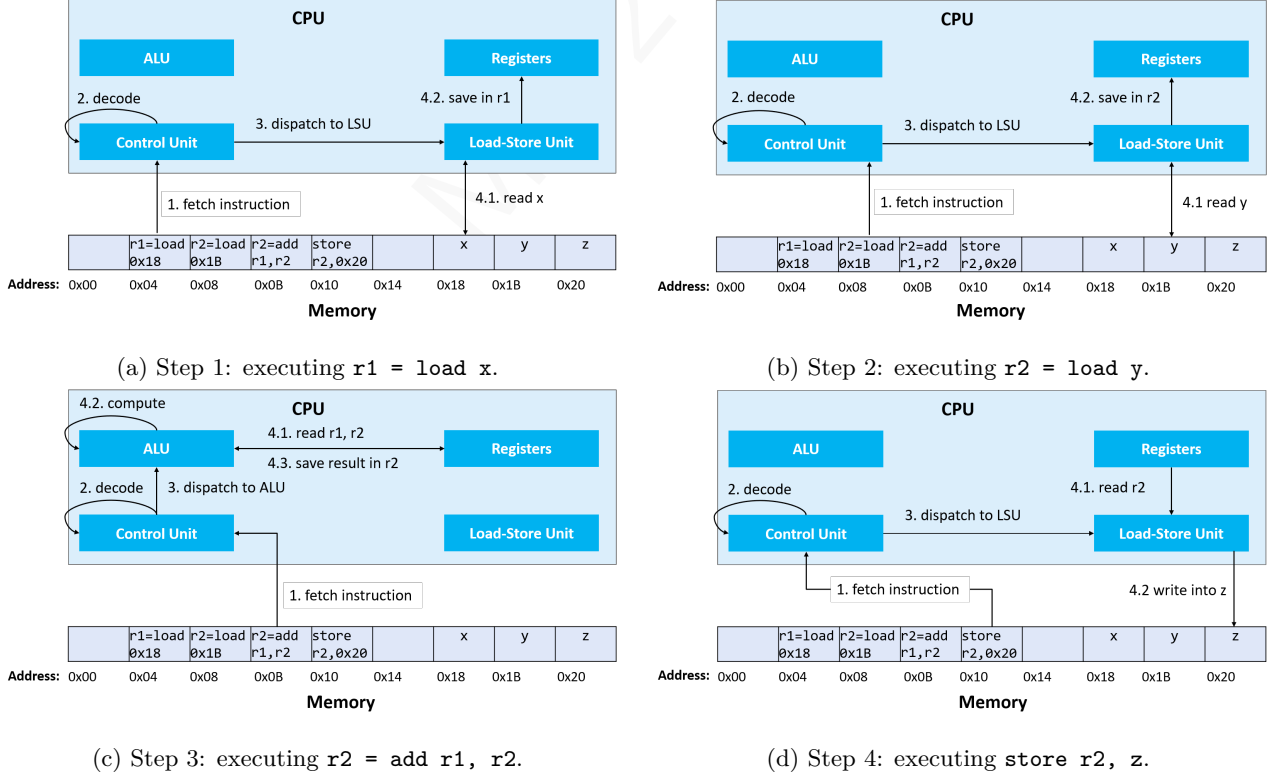


Figure 2.3: An example of executing four instructions for computing $z = x + y$.

2.4 Instruction Set Architecture (ISA)

Our previous example in Code 2.1 uses abstract representations of instructions. In practice, different processors support different instruction sets, and each instruction is encoded as binary data that the processor can decode and execute. The ISA defines the instructions that a processor can execute, including the specific instructions and registers available in the CPU. Since instructions are encoded in memory, how many bits are used for each instruction? Are they of fixed length or variable length? All these details are specified in the ISA documentation.

Different ISAs have different design details, each tailored to specific needs and trade-offs. The most commonly known ISAs include x86, x86-64, ARM, and RISC-V. x86 and x86-64 are complex instruction set computer (CISC) architectures, known for their rich instruction sets, where individual instructions can perform relatively complex operations. x86 traditionally refers to the 32-bit instruction set, while x86-64 extends it to support 64-bit computing, providing a much larger address space for modern systems. In contrast, ARM and RISC-V follow reduced instruction set computer (RISC) principles, featuring simpler and more regular instruction formats and operations.

In the following, we will mainly use ARM as an example to make the discussion more concrete. In general, an ISA defines several types of instructions, including *data movement*, *arithmetic and logic operations*, and *control flow*:

- *Data movement instructions*: Transfer data between registers and memory, or load immediate values into registers. For example:

```
mov x1, #10      ; x1 = 10
ldr x2, [x0]      ; Load data from memory into x2
str x2, [x0, #8]   ; Store data from x2 to memory
```

Here, `mov` loads the immediate value 10 into `x1`. The register `x0` contains a memory address. The `ldr` instruction reads the data at that address into `x2`, while `str` writes the value in `x2` to the memory address `x0 + 8`.

- *Arithmetic and bitwise instructions*: Perform arithmetic and bitwise operations on data. For example:

```
add x1, x0, x2    ; x1 = x0 + x2
and x1, x0, x2    ; x1 = x0 & x2
```

The `add` instruction adds two register values and stores the result in a register. The `and` instruction performs a bitwise AND operation on the two operands, producing a result in which each bit is set only when the corresponding bits of both operands are set. Other arithmetic instructions include `sub`, `mul`, and `udiv`, while other bitwise operations include `orr` and `eor`.

Thinking Question

In the previous lecture, we saw that addition and subtraction can be implemented using the same adder circuit. Why does an ISA provide separate `add` and `sub` instructions? Could a single instruction be sufficient?

Thinking Question

While integer division requires separate instructions for signed and unsigned operands (`sdiv` and `udiv`), addition and subtraction can use the same instructions for both signed and unsigned integers. Why?

- *Control flow instructions:* Change the sequence of program execution. For example:

```
cmp x0, #0      ; Compare x0 with 0
b.eq label     ; Branch to label if they are equal
```

The `cmp` instruction compares two values by internally computing their difference and updating the condition flags. The conditional branch `b.eq` checks these flags and transfers control to `label` when the two values are equal. Otherwise, execution continues with the next instruction. Other conditional branches include `b.ne`, `b.lt`, `b.le`, `b.gt`, and `b.ge`, while `b` performs an unconditional branch.

These instruction types provide the basic operations needed to execute programs. They can be combined to implement algorithms and program logic. Next, we use two examples to illustrate how these ARM instructions can be combined to implement useful algorithms.

```
sub x1, x0, #1
and x2, x0, x1
cmp x2, #0
b.eq is_power_of_2
is_not_power_of_2:
...
b done
is_power_of_2:
...
done:
...
```

Code 2.2: Checking if an integer is a power of two.

```
mov x1, #1
mov x2, #1
loop:
cmp x2, x0
b.gt done
mul x1, x1, x2
add x2, x2, #1
b loop
done:
...
...
```

Code 2.3: Computing the factorial of an integer.

Example 1: Power of Two. Code 2.2 demonstrates a program that checks whether a positive integer is a power of two using

$$n \& (n - 1) = 0.$$

Suppose `x0` contains an integer n . The program first computes $n - 1$ using `sub` and stores the result in `x1`. It then performs a bitwise AND between n and $n - 1$ using `and`, storing the result in `x2`. Next, `cmp` compares `x2` with zero and updates the condition flags. If they are equal, `b.eq` transfers control to `is_power_of_2`; otherwise, execution falls through to `is_not_power_of_2`. Finally, both paths branch to `done`.

Example 2: Factorial. Code 2.3 demonstrates how to compute the factorial of a given integer n . At a high level, the program repeatedly multiplies the current result by an increasing integer:

$$result = 1; \quad \text{while } i \leq n : \quad result = result \times i; \quad i = i + 1.$$

When the loop terminates, `result` contains $n!$.

In the assembly program, `x0` contains n , `x1` stores `result`, and `x2` stores i . The two `mov` instructions initialize `x1` and `x2` to 1. In each iteration, `cmp` compares i with n , and `b.gt` branches to `done` when $i > n$. Otherwise, `mul` multiplies the current result by i , `add` increments i by one, and `b` jumps back to `loop`. Thus, the backward branch implements the loop in the high-level program.

2.5 I/O and More

Besides performing computations, a computer needs to receive input from the external world and produce output. These functions are provided by I/O devices. I/O devices allow computers to interact with the external world, enabling data entry, display, communication, and storage.

In general, there are two ways for the CPU to communicate with I/O devices:

- *Memory-mapped I/O*: I/O device registers or frame buffers are assigned specific addresses within the system's memory space, allowing the CPU to use the same load and store instructions as it does for normal memory access to read from or write to these devices. Typical examples include VGA controllers (Figure 2.4a), disk controllers, and network interface cards.
- *Port-based I/O*: Also called isolated I/O, where I/O devices are accessed through a separate address space using special instructions (such as `in` and `out` on x86), which distinguish device operations from regular memory operations. Examples include legacy PC peripherals like the keyboard controller (Figure 2.4b) or some serial/parallel ports.

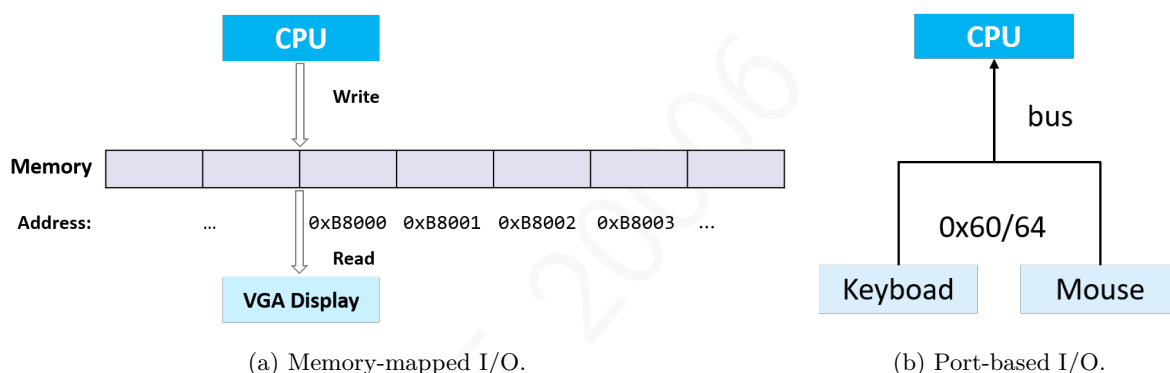


Figure 2.4: Two I/O modes.

Regardless of whether I/O devices are accessed through memory-mapped I/O or port-based I/O, a key question arises: when the CPU attempts to read data from a device, how does it know that the data is ready? Two common mechanisms are used to coordinate between the CPU and I/O devices:

- *Polling*: The CPU repeatedly checks a status register of the device until it indicates that data is ready. For example, a keyboard controller sets a “data ready” bit that the CPU must poll before reading the keystroke.
- *Interrupts*: The device sends an interrupt signal to the CPU when it is ready to be serviced, eliminating the need for constant polling. For instance, a network card generates an interrupt when a packet arrives, allowing the CPU to process it immediately.

For large data transfers, the CPU can rely on *Direct Memory Access (DMA)*, which allows the device to read from or write to main memory directly, bypassing the CPU. DMA is often combined with interrupts to notify the CPU when the transfer is complete. For example, a disk controller uses DMA to transfer disk blocks directly into RAM while the CPU continues executing other tasks.

Exercise

- 1) Suppose `x0` contains a positive integer n . What does the following code do? What does the value in `x1` represent?

```
    mov x1, #0
    mov x2, #2
loop:
    udiv x3, x0, x2
    mul x4, x3, x2
    cmp x4, x0
    b.eq not_p
    add x2, x2, #1
    cmp x2, x0
    b.lt loop
    b is_p
not_p:
    mov x1, #0
    b done
is_p:
    mov x1, #1
done:
    ...
```

Code 2.4: ARM assembly code.

- 2) Write an assembly program that computes the largest Fibonacci number less than a given threshold n . The Fibonacci sequence is defined as

$$F_0 = 0, \quad F_1 = 1, \quad F_k = F_{k-1} + F_{k-2} \quad (k \geq 2).$$

For example, when $n = 10$, the Fibonacci numbers less than n are 1, 1, 2, 3, 5, and 8, so the program should output 8.